Science et musique, Comment recréer le plus fidèlement possible une musique ?

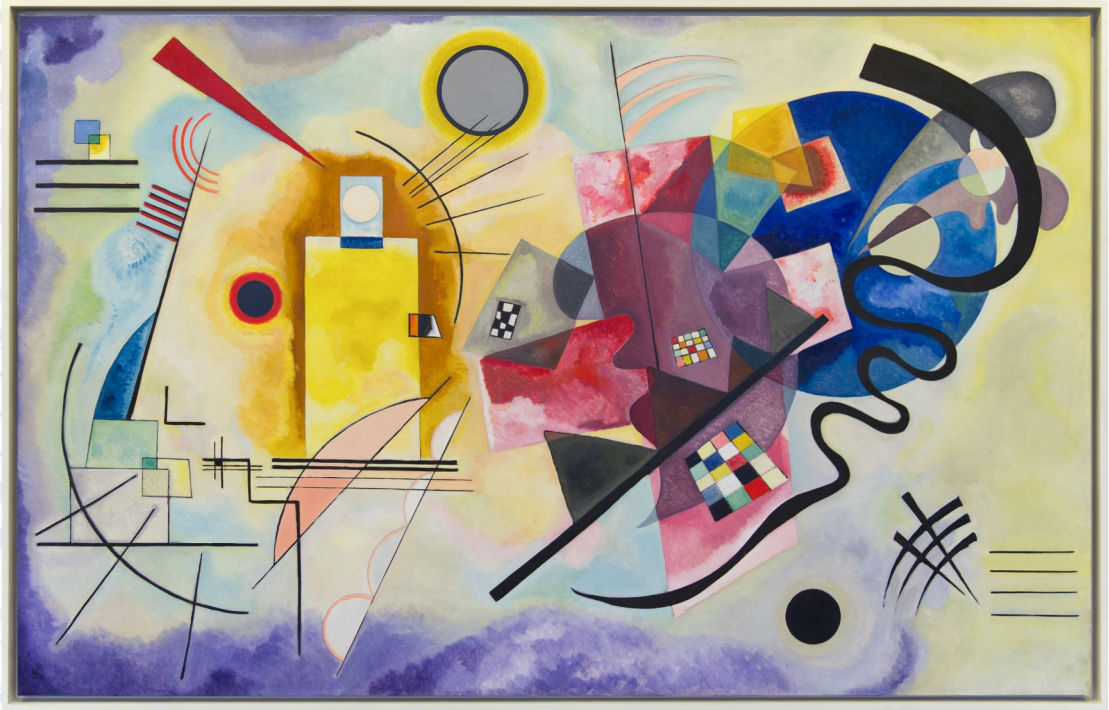


Figure 1: Wassily Kandinsky, Jaune-Rouge-Bleu (1925). Huile sur toile. Musée National d'Art Moderne, Centre Pompidou.

ANTRI BOUZAR Aslan

BROUSSARD Loup

DELAGNES Victor

OLIVE Lucie

VEYSSEYRE Clement

Tuteur : PANTEL Marc

TOULOUSE INP

prepa t²
Prépa Toulouse Transitions

Remerciements :

Tous nos remerciements à Mr. Pantel notre tuteur pour son soutien, ses réponses à nos questions et son aide dans ce projet.

Également à Mr. Pinaud, pour son implication dans la réalisation des projets, les ingénieurs pédagogiques de Toulouse INP et les documentalistes de la Bibliothèque Universitaire de l'ENSIACET pour leur encadrement dans les différents ateliers effectués au cours de ce projet.

Abstract

"Music is a hidden arithmetic exercise of the soul, which does not know that it is counting." >

— G. W. Leibniz (Letter to Christian Goldbach, April 17, 1712).

The awareness of links between music and science is an ancient preoccupation started by Pythagoras (c. 570 – c. 495 BC). This interest has never been abandoned by scientists throughout history. Music has been made and concurrently analysed with increasing precision culminating in advanced mathematical tools such as the Fourier transform. Currently, technological advances enable the emulation of classical instruments, and if this begins with the *CAM* (computer-assisted music), this continues with generative AI platforms such as *Suno*.

This report investigates a method to recreate music by analysing piano recordings. By applying the Fourier transform, acoustic signals are processed via a Python program to extract pure tones, before using them in another environment called Strudel to create music. This report details the research and the programs produced in both languages. This work is situated within the technical and cultural framework of Live Coding, demonstrating how algorithmic processing can contribute to open-source creative libraries.

Keywords: *Live Coding, Fourier Transform, Strudel, Python, Spectral Analysis, Open-Source*

Table des matières

Abstract **3**
Introduction **5**
Comment créer un son **5**
Perception du son **8**
Enregistrement et transformées de Fourier **10**
Code Python **12**
Algorave et Live Coding **15**
Quel message derrière cette pratique ? 15
De la théorie à la pratique : Initiation au Live Coding via Strudel **16**
Comment ça marche ? 17
Analyse réflexive du travail de groupe **18**
1. Dynamique de groupe et complémentarité des profils 18
2. Prise de recul face aux difficultés rencontrées 18
3. Amélioration continue et apports des ateliers Moodle 19
Références Bibliographiques **20**
Articles scientifiques 20
Livres 20
Thèses de doctorat 20
Documents trouvés sur internet (Vidéos, Articles web, Documentation technique) 21
Pages internet (Sources pédagogiques ou d'information générale) 22
Annexes **23**
Glossaire des termes techniques 23
Physique et Mathématiques de l'Acoustique 23
Informatique et Programmation 24
Musique et Production Musicale 25
Notebook interactif : Introduction pratique au Live Coding avec Strudel 26
Rythmes et patterns 26
Notes et monophonie 27
Accords et Renversements 27
Arrangement 27
Les Samples et leur Importation 28
La Méthode Locale 29
La Méthode en Ligne 30
Exemples de morceaux 32
Programme Python et tableaux, *fourier.py* 32
Relevé des décisions des réunions 36
Réunion 1 36
Réunion 2 37

Réunion 3.....39
Réunion 4.....41

Introduction

Les liens entre science et production musicale sont très nombreux et le choix d'un sujet s'est avéré difficile. La physique des ondes appliquée aux cordes ou les mathématiques des rythmes euclidiens, que Paul Lascabettes, chercheur à l'*IRCAM*, aborde sur son site[23], auraient pu être intéressant. Il était possible de parler du dimensionnement des salles de concerts, du métier d'ingénieur du son ou des explorations rythmiques menées par des musiciens tel que *Tigran Hamasyan*. Nous avons cependant décidé de parler d'une manière émergente de faire de la musique, le live coding. Nous allons vous présenter ce nouveau mouvement artistique en reprenant les bases de la musique, de la production et perception d'un son à la numérisation de celui-ci avant de pratiquer le live coding.

Comment créer un son

On ne peut pas commencer à parler de science et de musique sans expliquer ce qu'est un son et comment le produire.

« Un son est un ébranlement élastique de l'air, d'un fluide ou d'un solide qui se manifeste par des variations de pression autour de la pression moyenne du milieu »[4]. Pour faire plus simple, un son est une succession de vibrations qui se succèdent dans un milieu. Un son est une onde et peut donc être défini par une fréquence (nombre de vibrations par seconde) et une longueur d'onde (distance parcourue par l'onde pendant une période temporelle T). Il existe plusieurs types d'onde (électromagnétique, mécanique) mais un son est une onde acoustique que l' on peut entendre et donc dont la fréquence est comprise entre 20 Hz et 20 000 Hz.

Ci-contre, on peut voir un exemple de signaux sonores de périodes et de timbres différents.

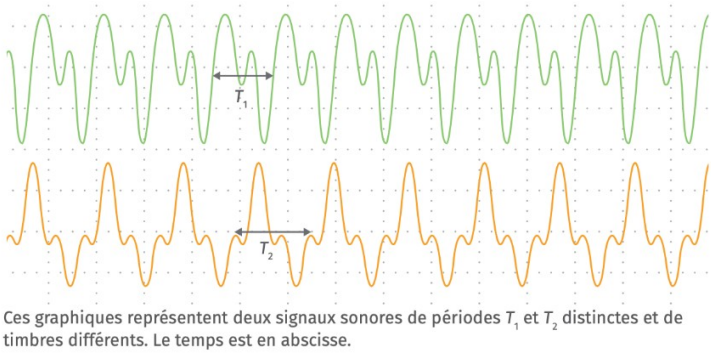


Figure 2: image issue du site livre scolaire [22]

Pour produire un son, il existe de nombreuses manières. Il est possible d'utiliser une multitude d'instruments différents. Il en existe à cordes, à vent, des cuivres ou encore des percussions. Le mode de fonctionnement est différent pour tous ces instruments. En effet, les instruments à cordes vont reposer sur le fait de faire vibrer une ou plusieurs cordes en les pinçant (guitare) ou en les frottant (violon avec un archet). Ces vibrations vont être amplifiées dans une caisse de résonance ce qui permet de porter le son. Ceux à vent sont totalement différents. Le musicien souffle dans l'instrument ce qui fait vibrer une colonne d'air, produisant un son. Pour les percussions le musicien frappe sur des objet pour produire des vibrations et donc un son. Tous ces instruments créent des sons possédant des caractéristiques distinctes. Lorsque l'on prend la note *La* d'un diapason, d'un piano ou d'un violon, on ne va pas obtenir exactement le même son. On peut expliquer cela en introduisant les notions de couleur et de timbre.

Lorsqu'un instrument produit une note, il émet un signal périodique mais peut être sinusoïdal ou plus complexe. C'est là que rentre en jeux le timbre des instruments ou la couleur du son. Évidemment, la note principale reste la même mais dans le cas d'un diapason, on peut voir que le signal est sinusoïdal.

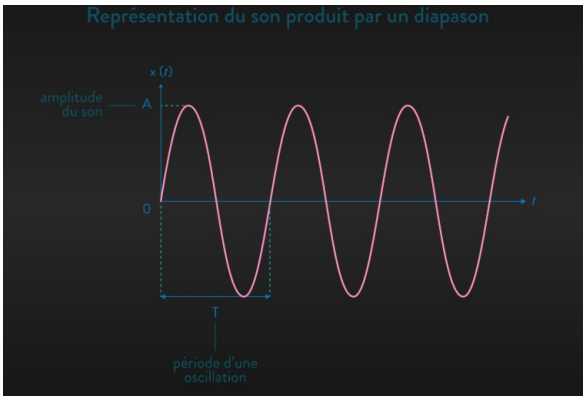


Figure 3: Représentation du son produit par un diapason [25]

Il n'y a qu'une seule fréquence et on peut donc dire que le son est pur ou qu'il n'a qu'une seule couleur.

On peut comparer ce signal à celui d'un piano que l'on a au préalable enregistré. On peut voir que le signal est bien moins pur et plus complexe que celui du diapason.



Figure 4: le spectre de la note de do (amplitude en fonction du temps), issue de l'application Audacity

Cela est dû à l'instrument avec lequel on joue. C'est son timbre. Toutes ces impureté, ces résonances font que les instruments ne sonnent pas exactement de la même façon pour la même note. Nous

pouvons analyser ce son à l'aide de la transformée de Fourier. « Un signal périodique de fréquence f et de forme quelconque peut être obtenu en ajoutant à une sinusoïde de fréquence f (fondamentale) des sinusoïdes dont les fréquences sont des multiples entiers de f » [10]

On peut donc comprendre que le son produit par un piano ou tout autre instrument de musique est une somme de différentes fréquences et donc de différentes notes. On peut voir sur le signal précédent que la fondamentale (note de do) est le pic le plus élevé et nous pouvons également remarquer des « impuretés » après.

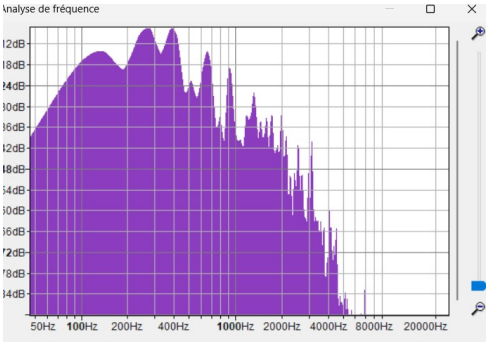


Figure 5: Graphique de répartition des fréquences en fonction des décibels, issue d'Audacity

Ce graphique est une répartition des fréquences en fonction des décibels. Le premier pic, le plus à gauche est la fréquence fondamentale, c'est la fréquence de la note de *Do*, la note que l'on perçoit. Les pics suivants sont les harmoniques ou les fréquences qui sont des multiples d'entier de la fondamentale. Lorsque le pianiste appuie sur la note de *Do*, il joue évidemment un *Do* mais aussi une multitude d'autres notes qui résonnent.

C'est cela qui fait le charme d'un instrument. Ces résonances sont intrinsèques à l'instrument. Un piano d'une autre marque ou fait avec un autre matériau va sonner et donc résonner différemment. On peut regarder la note de *Do* d'un violon par exemple et on verra que la fondamentale est la même mais que les couleurs sont différentes.

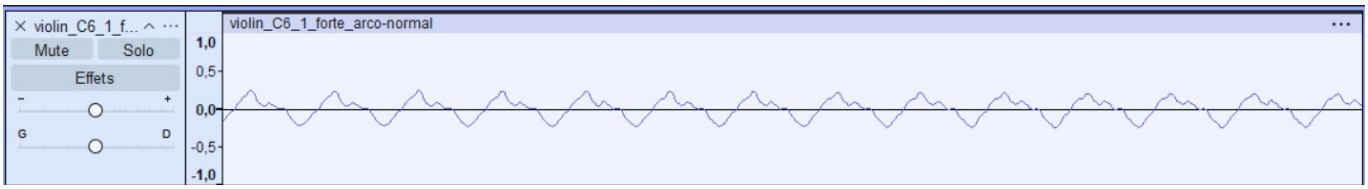


Figure 6: spectre de la note de do d'un violon, enregistré par un orchestre philharmonique [24], issues d'Audacity

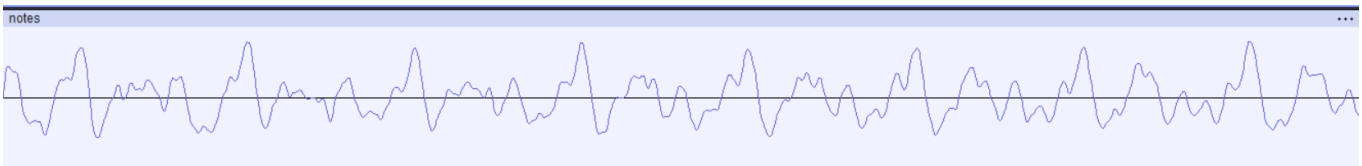


Figure 7: spectre de la note de do d'un piano, enregistré par Aslan Antri Bouzar, issues d'Audacity

Sur les deux spectres on retrouve en abscisse le temps et en ordonnée l'amplitude, cependant

l'échelle n'est pas la même. Les notes sont identiques, seules les poids des harmoniques changent. En effet, même si les deux instruments sont de la même famille, le principe du piano est de percuter une corde avec un marteau tandis que pour jouer du violon, il faut frotter un archer sur les cordes. Le matériau est également différent et la forme de l'instrument n'est pas la même. Tous ces facteurs font que le piano et le violon, pour une même note, ne font pas le même son et donc ne possèdent pas le même timbre.

Après avoir vu le fonctionnement général des instruments, et expliqué l'origine des différents timbres, on peut alors se demander comment percevons nous la musique et sommes nous objectifs vis-à-vis de celle-ci.

Perception du son

Nous allons d'abord chercher à comprendre pourquoi nous aimons écouter de la musique, particulièrement celle que nous connaissons et pourquoi est-ce stimulant intellectuellement. Cela repose sur un mécanisme de prédiction, de mémoire et de récompense : lorsqu'un auditeur écoute de la musique, son cerveau ne traite pas seulement le son de manière passive, il anticipe activement la suite. Étant familier à une forme de musique qu'il reconnaît (dans la structure, les rythmes et harmonies), le cerveau va essayer de prédire la suite, si la prédiction est correcte les circuits de la récompense s'activent.

Le plaisir intellectuel vient donc aussi du fait d'avoir raison sur sa prédiction, et non de la musique en elle-même [1][2]. Là est donc tout l'intérêt de la musique moderne et contemporaine, qui casse les codes. Au début elle paraît déstructurée, dénuée de sens mais au fur et à mesure d'une écoute attentive, le cerveau commence à pouvoir prédire la forme, l'harmonie, les rythmes et mieux l'apprécier. En résumé, une musique que nous connaissons bien va nous procurer du plaisir jusqu'à devenir trop prévisible pour notre cerveau.

Le cerveau a besoin de surprise sans être perdu sinon il perçoit la musique comme étant incompréhensible. Ainsi un style de musique trop différent du nôtre (musique de cultures différentes,

musique atonale, dodécaphonique) nous sera plus compliqué à apprécier. Cependant il ne faut pas en être repoussé car ils contiennent justement des richesses qui nous échappent et qui pourraient élargir nos horizons.

Un mélomane d'un style trop complexe pour l'oreille d'un néophyte a juste un cerveau qui comprend et prédit, de par son habitude, des harmonies et schémas musicaux plus complexes, qui lui apportent donc d'autant plus de plaisir.

Les détracteurs de la musique électronique jouée par ordinateur critiquent son aspect parfaitement régulier. Ce n'est pourtant pas un hasard, la perception musicale humaine est extrêmement sensible et friande de micro-variations de timing, articulation et timbre. Ces éléments, souvent appelés expressivités musicales ou imperfections, sont essentiels pour transmettre une intention ou une émotion. Une musique robotique sans retard ou groove, nous paraîtra donc bien trop fade et sans reliefs. L'utilisation d'enveloppes sur nos sons afin de créer une dynamique dans notre musique permet de masquer ce problème. Comme dit précédemment, le plaisir d'écouter de la musique vient aussi de son côté prédictible, une musique trop parfaite et robotique serait donc sans intérêt pour notre cerveau.

À l'inverse le phénomène de surprise, par rapport à toute chose et non seulement la musique est appelé codage prédictif. Par exemple, lorsque nous descendons des escaliers, le mouvement paraît instinctif mais dès que nous ratons une marche, nous sommes paniqué. Le cerveau ne s'attendait pas à cet écart, comme l'explique *Pang Zhaoyang* **:** «au lieu de recevoir passivement les informations externes pour former une perception ou une décision, le cerveau utilise un modèle interne hiérarchique qui pourrait interagir activement avec les stimuli externes. »[6] **.** C'est donc parce que le cerveau connaît le son d'un instrument, d'un morceau, d'un style qu'une version numérique peut manquer de « saveur ».

Ainsi notre perception du son n'est pas rationnelle et absolument cartésienne, il nous faut le prendre en compte. Un exemple de technologies contraint de s'adapter à notre perception du son, sont les écouteurs à conduction osseuse. Le son passant à travers de la matière et non de l'air sera perçu différemment, il y a donc des ajustements à effectuer sur ces écouteurs pour qu'on ne puisse percevoir cette différence.

Nous allons maintenant présenter notre démarche concrète en lien avec ce projet.

Enregistrement et transformées de Fourier

Nous allons présenter ici toutes les étapes par lesquelles nous sommes passés lors de nos recherches.

Tout d'abord nous avons enregistré, à l'aide d'un micro, d'une interface son et du logiciel *Reaper* ; un piano sur trois octaves note à note depuis le Do 3 (130,81 Hz) au Do 6 (1046,50 Hz). Nous avons redécoupé le signal obtenu pour isoler au format Wav chaque note.

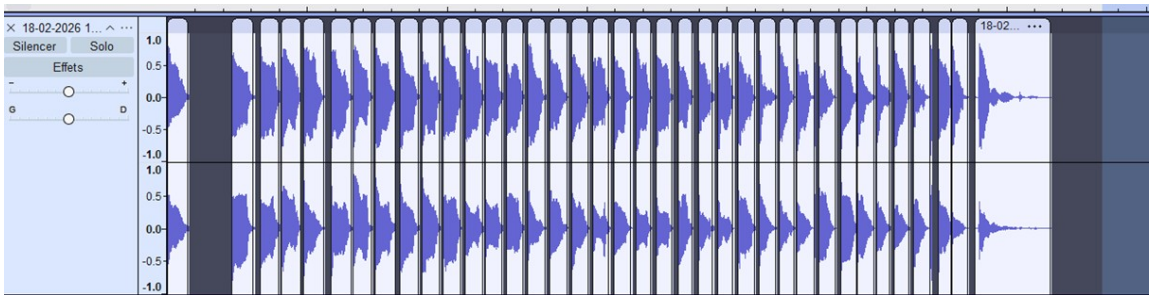


Figure 8: Capture d'écran Audacity, enregistrement coupé

Audacity permet alors de faire une analyse spectrale de chaque note, ce qui nous donne ceci :

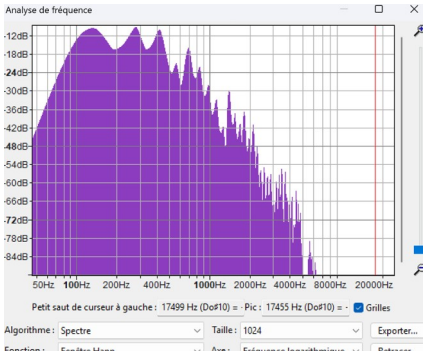


Figure 10: Tracé d'analyse spectrale (Audacity) d'une note

Fréquence (Hz)	Niveau (dB)
43.066406	-55.256332
86.132813	-43.770874
129.199219	-33.875858
172.265625	-16.445232
215.332031	-8.722349
258.398438	-12.519271
301.464844	-27.765558
344.531250	-31.375044

Figure 9: capture d'écran fichier txt téléchargé depuis Audacity

On obtient donc un tracé des décibels en fonction des fréquences émises par le son. En effet le signal est riche, constitué de nombreux harmoniques. Cependant nous cherchons ici à récupérer uniquement l'harmonique principal qui constitue notre son à l'état pur comme le jouerait un diapason par exemple. À partir de l'analyse spectrale d'*Audacity* nous pouvons alors exporter un fichier texte répertoriant les fréquences et décibels associés comme le montre la figure 10.

L'objectif est, au travers de script python, de récupérer la fréquence associée au plus haut niveau de décibel, autrement dit l'harmonique principal. Dans le cas d'un signal bruité, de faibles qualités cette technique s'avère peu efficace. Heureusement un outil dont les applications dépassent largement le cadre du traitement du signal va nous régler le problème, la Transformée de Fourier (TF).

Face à ce problème, nous pouvons nous pencher sur le fonctionnement de la TF. Cette fonction prend en paramètre un signal riche et le décompose en somme de sinusoïdes, chacune associées à une fréquence (on rappelle ici que la fréquence est l'inverse de la période T).

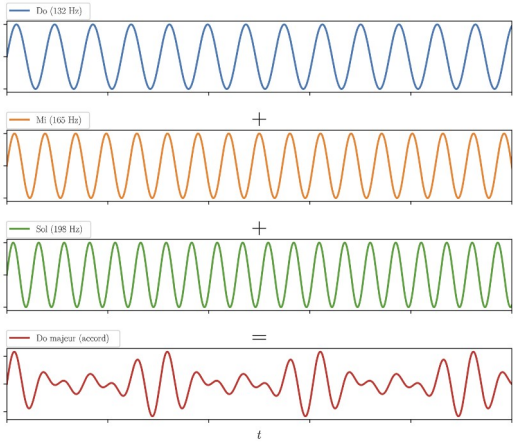


Figure 11: Image d'illustration TF [19]

$$x(t) = a_0 + \sum_{n=1}^{+\infty} a_n \cos\left(n\frac{2\pi}{T}t\right) + b_n \sin\left(n\frac{2\pi}{T}t\right)$$

Figure 12: Formule mathématique de la transformée de Fourier [19]

Nous pouvons ainsi récupérer les harmoniques qui nous intéressent en regardant les coefficients a_n et b_n que nous retournent la fonction. Le problème étant résolu sur le plan théorique, nous pouvons passer à l'implémentation en python de la TF.

Code Python

Nous avons essayé deux méthodes différentes, la première où le programme prend en paramètre un des fichiers .wav extrait d'*Audacity* et la deuxième exploitant l'analyse spectrale de ce même fichier par *Audacity*. À noter que les fichiers peuvent être composés de 2 pistes quand ils sont en stéréo, ce qui nous impose d'en choisir une des deux ici. On applique ensuite la fonction transformée de Fourier fft du module *scipy.fftpack* aux éléments de la piste choisie.

Comme les valeurs des éléments sur lesquelles on applique la transformée sont réelles, le tableau (array) obtenu est symétrique, il suffit donc de prendre la moitié du tableau. Il ne manque plus qu'à tracer le graphique obtenu et trouver les fréquences pour lesquelles les valeurs sont maximales.

Pour cela, on va utiliser le module matplotlib (plt), qui permet de tracer des fonctions. Commençons par configurer les paramètres généraux du graphique, comme sa taille et les axes. Traçons également un quadrillage pour avoir une meilleure idée de la valeur associée. On utilise ensuite la fonction *plt.plot* pour tracer la transformée de Fourier sur le quadrillage précédemment créé.

Pour finir, on cherche à récupérer la valeur de la fréquence pour laquelle le niveau est maximal. Pour cela il est nécessaire d'importer une fonction max gérant les objets de type *array*. En transformant *l'array* en *list*, on arrive à obtenir l'indice de la valeur max de la liste. Par la suite, on change la valeur de cet indice dans la liste par 0 et on cherche de la même manière le nouveau max.

Cependant, la valeur de l'indice donné ne correspond pas à la valeur visible sur le graphique. Dans le module *numpy (np)*, qui contient des fonctions relatives aux *arrays*, la fonction *np.argmax* renvoie l'indice du max du tableau. Mais au moment de faire les tests, si cette méthode correspondait pour le *Do 3*, elle ne renvoyait pas une valeur cohérente pour le *La 3* : un max visible à 500 Hz et une valeur de 333 Hz. Cette incohérence était due à la présence de la valeur absolue pour tracer le graphique, sans, le max visible est bien aux alentours de 330 Hz. Dans la librairie *matplotlib* il y a une catégorie/classe "*backend bases*" qui permet de réagir aux actions de la souris sur la fenêtre du graphique. Pour cela on utilise la fonction *plt.connect*, qui prend comme argument le type d'évènement et une fonction qui permet d'associer une action à cet évènement. Cela permet de créer un pointeur et de récupérer les fréquences associées aux maximums successifs.

Le résultat obtenu pour un *La 3* n'est pas cohérent ($f_{exp} = 333$ Hz $> f_{th} = 220$ Hz).

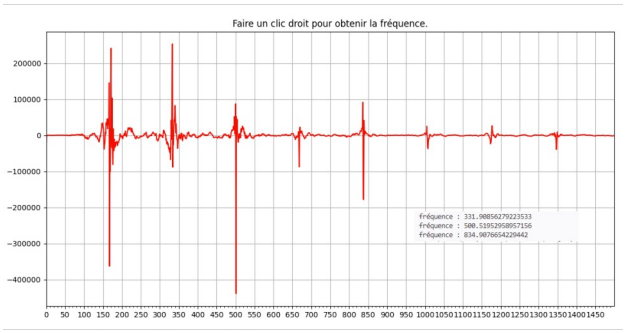


Figure 13: Capture d'écran de la réponse du programme contenu dans le fichier fourier.py en réponse au fichier contenant un La 3. On y voit le spectre obtenu et 3 fréquences obtenues après avoir cliqué sur les pics présents sur le spectre.

D'après Mr. Pantel, l'erreur viendrait du fait que l'échantillon fournît est trop court, et donc qu'il y aurait un facteur de multiplicité par rapport au résultat obtenu. L'échantillon fait 0.7 seconde, et en multipliant le 333 obtenu par 0.7, on obtient 233.1 ce qui est plus proche des 220 Hz attendus.

Dans le logiciel *Audacity*, il est possible d'analyser une sélection d'un fichier audio sous la forme d'un spectre, qui se traduit sous la forme d'un fichier texte quand on le télécharge. Celui-ci contient 2 colonnes, une avec les fréquences en Hz et l'autre avec le niveau sonore en dB. Le but ici est de récupérer le niveau sonore maximal et d'obtenir la fréquence associée, qui est celle de la fondamentale.

Fréquence (Hz)	Niveau (dB)
43.066406	-55.256332
86.132813	-43.770874
129.199219	-33.875858
172.265625	-16.445232
215.332031	-8.722349
258.398438	-12.519271
301.464844	-27.765558
344.531250	-31.375044

Figure 14: du fichier texte fourni par Audacity suite à l'analyse d'un la 3.

En premier, il faut ouvrir le fichier en mode lecture, avec la commande open en précisant 'r' pour read. Afin de trier les valeurs du fichier, on crée un dictionnaire, qui prend comme clé les fréquences et comme valeur le niveau sonore associé.

Par la suite, on parcourt le fichier ligne par ligne en conservant le contenu de chaque ligne dans une liste en séparant en fonction des espaces avec la fonction *.split* . Sinon, on récupère sous forme de réel les valeurs écrites, qui sont de base des chaînes de caractères, et on les insère dans le dictionnaire.

Une fois le dictionnaire rempli avec toutes les valeurs, on récupère le maximum des valeurs du dictionnaire, puis on regarde laquelle correspond au maximum précédemment récupéré.

Ainsi, on obtient la fréquence fondamentale. En itérant le processus n-fois et en veillant à retirer les valeurs extraites, on obtient les n-harmoniques de la note étudiée.

```python
def spectre():
    """
    analyser un spectre fait avec Audacity
    Audacity fournit un document texte avec 2 colonnes, une avec les fréquence, l'autre avec les nniveaux, et on cherche les
    maximums successifs
    """
    nombre=int(input("Combien d'harmonique voulez-vous ? "))
    nom_fichier = r'C:\Users\Lucie OLIVE\Documents\scolarité\prépa\2A\projet\spectre_la3.txt'
    #r avant la chaîne de caractères pour avoir une chaîne brute
    with open(nom_fichier, 'r') as fichier:
        tableau = {}   #dictionnaire : clé = fréquence, valeur : niveau sonore associé
        for ligne in fichier:
            contenu = ligne.split()     #sépare en fonction des espaces
            if len(contenu) != 2:   #on enlève la première ligne qui donne la légende du tableau
                pass
            else:
                freq=float(contenu[0])
                niv = float(contenu[1])
                tableau[freq]=niv
        max_tab = max(tableau.values())
        for indice in tableau:    #fonction enumerate
            if tableau[indice]==max_tab:
                print(f'Le niveau sonore maximal est de {max_tab} et il est atteint pour {indice} Hz, qui est la fondamentale.')
                tableau[indice] = -12345678912345
        for i in range(nombre):
            max_tab = max(tableau.values())
            for indice in tableau:
                if tableau[indice]==max_tab:
                    print(f"L'harmonique numéro {i+1} est atteinte pour {indice} Hz.")
                    tableau[indice] = -12345678912345
```

Figure 15: Capture d'écran du fichier spectre.py contenant le programme spectre décrit ci-dessus qui analyse le fichier fourni par Audacity pour trouver les fondamentales et les harmoniques suivantes.

```
Combien d'harmonique voulez-vous ? 4
Le niveau sonore maximal est de -8.722349 et il est atteint pour 215.332031 Hz, qui est la fondamentale.
L'harmonique numéro 1 est atteinte pour 430.664063 Hz.
L'harmonique numéro 2 est atteinte pour 258.398438 Hz.
L'harmonique numéro 3 est atteinte pour 473.730469 Hz.
L'harmonique numéro 4 est atteinte pour 645.996094 Hz.
```

Figure 16: Capture d'écran de la réponse du programme contenu dans spectre.py avec les 4 harmoniques suivant la fondamentale, suite à l'analyse du fichier fourni par Audacity pour un la 3.

Quand on a testé avec le spectre issu du *la 3*, les fréquences obtenues étaient plus crédibles que celles avec l'autre programme : on obtient 215 Hz pour la fondamentale, au lieu des 220 Hz attendus, ce qui reste qu'en même plus proche que les 500 Hz obtenus avec le premier programme.

Les différences de valeurs faites avec les valeurs théoriques peuvent être expliquées par la qualité de l'enregistrement ainsi que celle du piano utilisé, ou par des bruits parasites.

Algorave et Live Coding

L'ALGORAVE est certainement l'expression la plus claire du lien entre informatique et art. Ce mouvement musical est né au Royaume-Uni en 2011, étant au départ seulement le nom qu'Alex McLean et Nick Collins donnent à une rave organisée dans Londres. Néanmoins ce mouvement culturel a mis du temps à se définir, en 2004 Alex Mclean parlait déjà dans un article « *Hacking Perl in Nightclubs »[14]* du lien entre code et musique.

Il est important de noter que l'algorave est un mouvement musical et pas un genre, et ce mouvement peut être décrit comme un point de rencontre entre la philosophie des Hackers[3] et la culture du Clubbing et des raves.

Ce mouvement bien que contestataire repose sur des règles claires :

- Exposer les processus algorithmiques (coder en live et afficher son écran au spectateur)
- Utiliser des logiciels open sources
- Se méfier des institutions
- Effondrement des hiérarchies (pas de tête d'affiche aux événements)
- Respect des autres communautés
- Diversité des programmations et des publics

Quel message derrière cette pratique ?

Nous pouvons mentionner *Renick Bell* musicien, programmeur et professeur américain basé à Saigon, au Vietnam, à l'origine des premières algoraves en Asie, interrogé sur sa vision du live coding et de l'algorave dans un documentaire *Arte*[9], "*J'espère que ça motivera le public à être plus critique de leur relation avec les algorithmes et je veux qu'il comprenne que ces algorithmes ont été fabriqués*

par des humains [...] il y a des ingénieurs derrière ces algorithmes et que c'est à eux de rendre des comptes.".

On voit que reprendre le contrôle sur le code et les algorithmes est une manière de critiquer les géants de la tech « *GAMAM* ». Le mouvement s'inscrit dans une logique anticapitaliste et a été créé à l'origine avec ce message, mais maintenant le mouvement a pris une ampleur suffisante pour que certaines personnes pratiquent le live coding sans réelle revendication politique derrière.

Un message conjoint au premier et qu'on retrouve dans *L'Éthique des hackers*[3], "*il est possible de créer de la beauté avec des ordinateurs*", en cela l'algorave s'inscrit comme héritier direct de cette proposition pour introduire un nouveau type de relation avec l'informatique. Mais plus qu'une philosophie, ce mouvement s'incarne dans des outils concrets.

En conclusion, *David Bola* dans « *Algorave : la teuf en open source | Tracks | ARTE* » [9] nous dit : "*Créer de la beauté avec du code et des algorithmes, c'est donc un acte politique par essence qui détourne des outils pensés pour le travail et le commerce de leur fonction. Cet enjeu revient sur la table à chaque révolution informatique, la dernière en date étant bien évidemment l'IA. Et ce n'est pas tout : en investissant le live coding ou en détournant d'autres outils, on pourrait bien aboutir à des révolutions esthétiques.*"

Renick Bell l'explique ainsi : "*[L'objectif est d'amener le code] au même niveau qu'une batterie ou une guitare, où les gens vous considèrent simplement comme un musicien. Le message de la musique n'est pas la technologie, mais le message de la musique, c'est en quelque sorte la musique elle-même.*"

De la théorie à la pratique : Initiation au Live Coding via Strudel

Vous l'aurez compris, cette mouvance artistique, bien qu'existante depuis une quinzaine d'années, est restée underground jusqu'à maintenant. Cependant si nous vous en parlons ici c'est grâce aux algorithmes des réseaux sociaux qui depuis quelques mois ont propulsé le contenu de certains acteurs du milieu comme *DJ Dave* ou *Switch Angel* sur nos fils

d'actualités. En effet, la popularité grandissante de ces acteurs attire un nouveau public et la démultiplication des ressources disponibles pour apprendre.

Comment ça marche ?

La première étape consiste à choisir notre interprète qui nous permettra à partir de langages tels que *Python*[26] de produire notre musique. Pour ne citer que les outils les plus connus on a *TidalCycles*[28] avec des implémentations web comme *Strudel.cc*[27] et des outils français existent également qu'on peut retrouver sur *livecoding.fr*[21].

Une fois notre choix fait, il reste à apprendre comment utiliser notre outil, fort heureusement et toujours dans une idée de partage il existe énormément de ressources en ligne fournies par des live codeurs plus expérimentés.

Strudel s'est imposé parmi les néophytes pour sa simplicité d'utilisation (une page web) et sa documentation claire et précise. *Strudel.cc*[27] est un outil open source pour la création musicale basée sur JavaScript et inspirée de *TidalCycles*[28] et développée en 2022. La possibilité de modifier et d'ajouter de nouvelles fonctionnalités a grandement accéléré le développement de Strudel qui regorge de possibilités. Nous avons décidé d'en explorer une partie lors de ce projet. Pour des raisons pédagogiques et afin de rendre la découverte de cet outil plus agréable, nous vous invitons à découvrir notre travail sur l'utilisation de Strudel au travers d'un notebook interactif *Observable*. Il est conseilliez de vous munir d'un casque ou d'un système de son approprié pour profiter pleinement de la suite.

N.B sur l'utilisation d'Observable : le choix du notebook interactif s'est imposé naturellement, il permet non seulement de s'affranchir de toute installation technique en exécutant le JavaScript directement dans le navigateur, mais il restitue surtout l'essence même du Live Coding en offrant au lecteur la possibilité d'expérimenter et de modifier la musique en temps réel.

Accès au notebook :https://observablehq.com/d/10a7e22d37d67443

Analyse réflexive du travail de groupe

Conformément à la démarche présentée lors de nos enseignements, nous avons mené une réflexion sur notre méthode de travail. Cette relecture s'est avérée être une source d'amélioration continue pour mener à bien ce projet pluridisciplinaire.

1. Dynamique de groupe et complémentarité des profils

Dès le début du projet, notre première action a été d'identifier les forces de chacun pour nous répartir efficacement les tâches. La présence d'Aslan a été un atout majeur pour la prise de son et l'approche théorique de la musique. Lucie et Clément se sont concentrés sur la modélisation mathématique et l'implémentation Python, tandis que Loup et Victor ont pris en charge le développement sur Strudel et la recherche sur l'esthétique Algorave. Enfin notre diversité de profils a été notre plus grande force, nous permettant d'aborder notre problématique sous un prisme à la fois artistique, physique et informatique.

2. Prise de recul face aux difficultés rencontrées

La démarche réflexive consiste à mobiliser sa pensée face à une situation complexe pour en tirer des leçons transférables. Nous avons été confrontés à deux obstacles majeurs qui ont nécessité une prise de recul :

- ***L'obstacle mathématique et algorithmique :*** *lors de l'analyse spectrale sous Python, notre algorithme renvoyait une fréquence fondamentale erronée. L'intervention de notre tuteur, M. Pantel, nous a fait réaliser que notre échantillon temporel de 0,7 seconde induisait un facteur de multiplicité. Cette erreur nous a rappelé l'importance de toujours confronter un résultat brut informatique à la théorie physique attendue.*
- ***L'obstacle technique d'intégration :*** *l'impossibilité de lire nos fichiers locaux .wav sur la version web de Strudel nous a d'abord bloqués. Au lieu de rester sur une solution locale (Node.js) qui entravait le partage collaboratif, nous avons su adapter notre approche en migrant nos ressources sur un serveur GitHub Pages.*

3. Amélioration continue et apports des ateliers Moodle

La séance de réflexivité et les ateliers ont aidé à notre structuration. Les outils et méthodes vus lors de ces séances nous ont incités à instaurer un système de réunions régulières, avec des relevés de décisions systématiques (joints en annexe). Si nous devions formuler des propositions d'amélioration pour un futur travail en équipe, nous mettrions en place plus tôt un outil de versioning commun (comme Git) non seulement pour les samples audio, mais aussi pour nos scripts Python, afin d'éviter la dispersion des données. De plus, une meilleure anticipation en laissant le temps de comprendre la physique de l'onde avant de vouloir la coder aurait fluidifié notre calendrier. Finalement, cette démarche nous a prouvés qu'analyser et comprendre nos erreurs de parcours sont une compétence fondamentale.

Références Bibliographiques

Articles scientifiques

1. Li, Q. ; Liu, G. ; Zhang, Y. ; Wu, J. ; Huang, R. Neural correlates of musical familiarity: a functional magnetic resonance study. *Cerebral Cortex* 34(4):bhae177, https://doi.org/10.1093/cercor/bhae177.

2. Robaczewski, A. ; Harkins, E. ; Pelland-Goulet, P. ; Gosselin, N. How Familiarity, Musical Affinity, and ADHD Shape Adolescents' Perception of Musical Emotions. *Behavioral Sciences* 15(11):1570, https://doi.org/10.3390/bs15111570.

Livres

3. Levy, S. *L'Éthique des hackers* (titre original : *Hackers: Heroes of the Computer Revolution*) ; Globe : Paris, 2013.

4. Tisserand, E. ; Pautex, J.-F. ; Schweitzer, P. *Analyse et traitement des signaux : Méthodes et applications au son et à l'image* ; Dunod : Paris, 2004.

Thèses de doctorat

5. McLean, C. A. *Artist-Programmers and Programming Languages for the Arts*. Ph.D. Thesis, Goldsmiths, University of London, Londres, Royaume-Uni, 2011. https://research.gold.ac.uk/id/eprint/6611/ (consulté le 12 avril 2026).

6. Pang, Z. *Codage prédictif dans le cerveau et les réseaux de neurones profonds*. Thèse de doctorat, Université de Toulouse, Toulouse, France, 2022. https://theses.fr/2022TOU30037 (consulté le 9 avril 2026)

Documents trouvés sur internet (Vidéos, Articles web, Documentation technique)

7. 3Blue1Brown. But what is the Fourier Transform? A visual introduction. https://youtu.be/spUNpyF58BY (consulté le 11 avril 2026).

8. Algorave Community. Algorave guidelines. https://github.com/Algorave/guidelines/blob/master/README_en.md (consulté le 11 avril 2026).

9. ARTE Tracks. Algorave : la teuf en open source. https://youtu.be/Crz6R4p_owI (consulté le 11 avril 2026).

10. Colaux, J. Le timbre des instruments ou la couleur du son. https://www.youtube.com/watch?v=TGUHbh6yYqY (consulté le 2 avril 2026).

11. Deezer / Ipsos. Étude IA & Musique. https://newsroom-deezer.com/fr/2025/11/deezer-ipsos-etude-ia-musique/ (consulté le 2 avril 2026).

12. Le Monde. Musique IA: Le business secret qui va casser le streaming. https://www.youtube.com/watch?v=G3d8wBOLS2c (consulté le 2 avril 2026).

13. Matplotlib Documentation. Mouse move and click event. https://matplotlib.org/stable/gallery/event_handling/coords_demo.html (consulté le 5 mars 2026).

14. McLean, A. Hacking Perl in Nightclubs. https://www.perl.com/pub/2004/08/31/livecode.html/ (consulté le 11 avril 2026).

15. SciPy Documentation. Fourier Transforms (scipy.fft). https://docs.scipy.org/doc/scipy/tutorial/fft.html (consulté le 19 février 2026).

16. Stack Overflow. Python Scipy FFT wav files. https://stackoverflow.com/questions/23377665/python-scipy-fft-wav-files (consulté le 19 février 2026).

17. Wikipedia. Algorave. https://fr.wikipedia.org/wiki/Algorave (consulté le 11 avril 2026).

Pages internet (Sources pédagogiques ou d'information générale)

18. http://algorave.com (consulté le 11 avril 2026)

19. *https://glq2200.clberube.org/chapitres/docs/signal-fourier* (consulté le 13 avril 2026)

20. https://www.haskell.org/ (consulté le 14 mars 2026)

21. http://livecoding.fr (consulté le 17 mars 2026).

22. Lelivrescolaire.fr. https://www.lelivrescolaire.fr/page/6225294 (consulté le 22 mars 2026).

23. https://www.mathemusique.fr/ (consulté le 13 avril 2026)

24. Philharmonia. Sound Samples. https://philharmonia.co.uk/resources/sound-samples/ (consulté le 12 avril 2026).

25. SchoolMouv. Le son, phénomène vibratoire (fiche de cours). https://www.schoolmouv.fr/cours/le-son-phenomene-vibratoire/fiche-de-cours (consulté le 12 avril 2026).

26. *https://docs.python.org/fr/3/* (consulté le 14 mars 2026)

27. https://strudel.cc/workshop/getting-started/ (consulté le 12 avril 2026).

28. *https://tidalcycles.org/* (consulté le 14 mars 2026)

Annexes

Glossaire des termes techniques

Afin de faciliter la lecture de ce rapport à un public néophyte, le lecteur trouvera les définitions des principaux termes techniques abordés dans le document.

Physique et Mathématiques de l'Acoustique

- **Transformée de Fourier :** la transformée de Fourier est un outil d'analyse des signaux musicaux. En particulier, elle permet d'obtenir une représentation Temps-Fréquence d'un enregistrement musical, appelée spectrogramme. En pratique, le spectrogramme est généré avec la TFCT (Transformée de Fourier à court terme) calculée sur des segments successifs de signal.

- **Fréquence fondamentale :** en acoustique musicale, la fréquence fondamentale est l'inverse de la période d'un son complexe. Cette fréquence est corrélée avec la hauteur du son musical.

- **Harmonique :** un harmonique (ou partiel harmonique) est en acoustique une des composantes d'un son complexe. Dans la nature, lorsqu'un corps sonore (telles qu'une corde ou une membrane) vibre, une fréquence fondamentale est émise. Des harmoniques sont également émis par résonance de cette fondamentale. Notez que la fréquence d'un partiel harmonique est nécessairement un multiple entier de la fondamentale.

- **Spectre sonore (ou Analyse spectrale) :** représentation graphique d'un son permettant de voir toutes les fréquences qui le composent (en abscisse) et leur intensité respective (en ordonnée).

- **Hertz (Hz) :** le hertz, de symbole Hz, est l'unité de fréquence du Système international (SI). Un hertz est la fréquence d'un événement qui se répète une fois par seconde : $1\ Hz = \frac{1}{s}$.

- **Décibel (dB) :** unité de mesure relative exprimant l'intensité (le volume) d'un son.

Informatique et Programmation

- **Array (tableau) :** en programmation (notamment en Python), il s'agit d'une structure de données permettant de stocker une liste de valeurs.

- **Open Source :** la désignation *open source* ou code source ouvert, s'applique aux logiciels (et s'étend maintenant aux œuvres de l'esprit) dont la licence respecte des critères précisément établis par l'Open Source Initiative, c'est-à-dire les possibilités de libre redistribution, d'accès au code source et de création de travaux dérivés. Mis à la disposition du grand public, ce code source est généralement le résultat d'une collaboration entre programmeurs.

- **GitHub / Dépôt (repository) :** GitHub est une plateforme en ligne très utilisée par les développeurs. Un "repository" (ou dépôt) est un espace de stockage sur cette plateforme où l'on range les fichiers d'un projet pour les sauvegarder, les gérer et les partager avec d'autres.

- **Serveur local (localhost) :** dans le domaine des réseaux informatiques, localhost (hôte local en français) est le nom habituel qui désigne une interface logique de l'ordinateur local.

Musique et Production Musicale

- **Live Coding :** pratique artistique consistant à écrire et modifier du code informatique en temps réel, devant un public, pour générer de la musique ou des visuels.

- **Algorave :** mot-valise fusionnant "Algorithme" et "Rave". Ce sont des évènements où la musique est entièrement générée en direct par des programmeurs (Live codeurs).

- **Sample (échantillon) / Sampling :** extrait sonore pré-enregistré (une voix, un coup de batterie, une note de piano) que l'on isole pour le réutiliser, le modifier et l'intégrer dans une nouvelle composition musicale.

- **Enveloppe sonore :** évolution de l'amplitude (le volume) d'un son dans le temps, depuis le moment où il est joué jusqu'au moment où il s'éteint. Elle comprend souvent l'Attaque, le Déclin, le Maintien et le Relâchement (ADSR).

- **BPM (Battements par Minute) :** unité de mesure du tempo d'un morceau de musique, indiquant sa vitesse.

Notebook interactif : Introduction pratique au Live Coding avec Strudel

Voici une restitution du texte du notebook :

Bienvenue sur notre notebook Strudel. Le but de celui-ci est d'introduire suffisamment de notions pour vous permettre de comprendre notre travail. Nous vous invitons à exécuter les cellules au fur et à mesure de la lecture afin de découvrir et d'écouter ce langage. La première étape, lorsque l'on s'apprête à jouer un morceau en groupe, est de déterminer un tempo. Le décompte du batteur que l'on entend sur certains enregistrements est généralement suffisant. Sur Strudel, on peut utiliser la fonction ci-dessous. Le premier paramètre est le bpm (que Strudel fixe à 120 si on ne précise rien) et le second la signature rythmique (4/4 généralement ou 3/4 pour les valses). Si vous exécutez la cellule, vous n'entendrez rien bien sûr, mais nous avons la première brique !

[Voir boîte de code à exécuter dans le notebook]

Rythmes et patterns

L'instrument le plus simple pour commencer est peut-être la batterie. Comme vous pouvez le voir ci-dessous, il existe des abréviations communes à tous les samples de batterie présents dans Strudel (et même au-delà dans le monde de la musique) pour nommer les différents éléments d'une batterie. *[Voir image : "drumset.png" dans le notebook] Image originale de Pbroks13*

Ici vous pouvez entendre différentes manières d'écrire le même pattern de batterie : la première est la plus synthétique, mais il est difficile de visualiser le pattern. La deuxième ressemble à une partition où chaque ligne correspond à un élément. La troisième permet de désolidariser chaque partie pour affiner le mixage. Afin d'activer ou non chaque partie, enlevez ou ajoutez _ devant le $ et appuyez

sur play (ou ctrl+enter) pour relancer, enfin appuyez sur stop (crtl+.) pour arrêter la cellule. *[Voir boîte de code à exécuter dans le notebook]*

Notes et monophonie

Maintenant que l'on sait programmer une batterie, on peut s'intéresser aux notes. Il existe, comme pour les rythmes, de très nombreuses manières d'écrire une mélodie. En voici quelques-unes : *[Voir boîte de code à exécuter dans le notebook]*

Accords et Renversements

Une mélodie ou une ligne de basse s'inscrit dans un contexte harmonique imposé par l'harmonisation de la gamme utilisée, et donc par des accords. Strudel possède un dictionnaire d'accords et différentes manières de renverser ces accords, c'est-à-dire d'empiler les notes qui le constituent, mais les possibilités restent limitées. Heureusement on peut résoudre ce problème, encore une fois, de plusieurs façons. *[Voir boîte de code à exécuter dans le notebook]*

Arrangement

Vous avez pu constater dans les précédentes parties que pour activer une piste, il fallait utiliser le symbole $ et le précéder de _ pour l'arrêter. Cette méthode permet d'écouter en parallèle plusieurs pistes et est beaucoup utilisée dans l'Algorave, puisque cette musique se base sur l'ajout progressif de pistes comme le fait un DJ avec des platines. Cependant, cette technique peut s'avérer difficile puisqu'il faut exécuter update (ctrl+enter) la cellule au bon moment. Comme nous souhaitions recréer des musiques de manière fidèle et ne pas avoir à activer/désactiver sans arrêt telle ou telle piste, nous avons choisi d'utiliser les fonctions arrange et stack : stack prend en paramètre des fonctions (let f =...) juxtaposées. arrange prend en paramètre une juxtaposition de : [nombre de

mesures, stack(f1, f2, ...)]. On peut indiquer une structure clairement. Si on applique cela à ce que nous avons fait précédemment, voici ce que l'on obtient : *[Voir boîte de code à exécuter dans le notebook]* Vous aurez sûrement tous reconnu *Billie Jean* de Michael Jackson, mais pourtant nous pouvons encore améliorer le résultat en :

- Affinant le mixage (les effets, les filtres, les enveloppes), cela demande une oreille très fine et une connaissance poussée des réglages et de leur impact sur le mix.

- Écoutant plus attentivement le morceau et rajoutant les petites parties qui manquent à notre introduction (une cabassa qui double le charley par exemple).

- Important des samples provenant directement de l'original comme par exemple la grosse caisse ou la caisse claire disponibles ici. Cette dernière idée nous mène tout droit à l'avant-dernière partie...

Les Samples et leur Importation

> "In sound and music, sampling is the reuse of a portion (or sample) of a sound recording in another recording." *(En musique, le sampling est la réutilisation d'une partie (ou d'un échantillon) d'une prise de son dans une autre prise de son.)* — **Wikipedia - Sampling (music)**

Strudel donne par défaut accès à des milliers de samples dont certains ayant marqué des époques comme le Roland TB-303, un synthé mythique qui définira l'acid house pour ne citer qu'un exemple.

Mais dans un souci d'originalité ou de perfectionnisme, on peut être amené à créer nos propres samples.

Sur la méthode de sampling que nous avons utilisée, nous vous renvoyons à la partie **Enregistrement et transformées de Fourier** de notre rapport de projet. Une fois notre sample créé, reste le souci de son utilisation. Comme souvent dans l'informatique récente, le souci n'est pas dans l'écriture de l'algorithme mais dans la configuration de l'environnement. L'importation de samples sur Strudel n'échappe pas à cette règle. Strudel nous donne le choix entre plusieurs méthodes ayant toutes leurs avantages et inconvénients : un environnement de "développement" en local**. U**n environnement de "production" en ligne.

La Méthode Locale

Strudel tourne dans un navigateur web en JavaScript, il ne peut pas directement lire les fichiers présents sur la mémoire locale de notre machine. Historiquement, le JavaScript est un langage qui servait seulement pour le web, on ne pouvait pas fouiller dans les dossiers de notre machine (sinon n'importe quel site web pourrait lire nos fichiers locaux). Pour résoudre ce problème, il nous fallait créer un pont entre notre disque dur et le navigateur. C'est exactement le rôle qu'a joué **Node.js**. Cet outil permet de faire tourner du code JavaScript directement sur notre machine. Nous l'avons utilisé pour lancer un serveur local en arrière-plan qui nous permet d'aller lire nos fichiers .wav dans Strudel, sans enfreindre les règles de sécurité du web. *(Si vous souhaitez plus de détails sur l'installation de cet environnement selon votre système d'exploitation, nous vous invitons à consulter la documentation officielle sur le site de Node.js).* Une fois l'installation réalisée, il est maintenant possible d'exécuter cette ligne dans un terminal : npx @strudel/sampler

Ce que fait cette commande : elle télécharge et lance temporairement l'outil Strudel Sampler. Elle crée un mini-serveur web privé sur le port 5432. Elle scanne automatiquement notre dossier (attention à l'emplacement) pour générer la liste de nos samples, permettant à Strudel de les charger instantanément via l'adresse http://localhost:5432.

Comme dit précédemment c'est une méthode locale, à laquelle on peut trouver certains défauts... Le plus criant étant l'absence de portabilité. La musique ne fonctionne que sur l'ordinateur qui fait tourner le serveur en local. Ce qui est contradictoire avec l'idée de partage de la communauté Algorave. L'idée ici n'est pas de fournir un manuel d'utilisation entier sur l'implémentation de samples dans le live coding mais de montrer par cet exemple que des barrières techniques telle qu'une dépendance au terminal ou un conflit de ports amènent à des problèmes d'idéologie vis-à-vis d'une communauté, un schéma qu'on applique ici à l'informatique mais visible tant dans les maths que dans la physique. Bien que parfaite pour la phase créative, cette méthode locale présentait trop de barrières pour un partage public. Pour que notre morceau soit écoutable par tous, directement dans ce navigateur, il nous a fallu migrer vers une solution d'hébergement permanente : GitHub Pages.

La Méthode en Ligne

Arrive le passage en production. Comment passer d'un environnement de "brouillon" (local) à un environnement de "production" (en ligne et public) pour que le lecteur puisse jouer les samples directement ? D'une réflexivité sur un projet annexe par des membres du groupe suivant le parcours Maths-Info est venue l'idée de l'outil GitHub.

"GitHub (/ɡɪthʌb/, entreprise GitHub, Inc.) est un service web d'hébergement et de gestion de développement de logiciels, utilisant le logiciel de gestion de versions Git." — **Wikipedia - GitHub**

La démarche technique se décompose ainsi : créer un dépôt (repository) public sur Github. Déposer/téléverser tous nos fichiers audio .wav/.mp3 finaux. Dans les paramètres du dépôt, activer "GitHub Pages". C'est cette action précise qui transforme un simple dossier de stockage en un serveur web afin de distribuer des fichiers. Dans le code Strudel, remplacer toutes les adresses temporaires (http://localhost:5432/X.wav) par les adresses publiques et permanentes (https://name_repository/path).

En passant par cette méthode, on résout alors le problème de portabilité, les samples fonctionnent pour n'importe qui, sur n'importe quelle machine, sans aucune installation requise. Ainsi qu'une disponibilité permanente, plus besoin de garder un terminal ouvert sur machine. Les fichiers sont hébergés dans le Cloud par les serveurs de Microsoft/GitHub. Ci-dessous un exemple d'utilisation : *[Voir boîte de code à exécuter dans le notebook]*

Pour la petite histoire : ce rythme de batterie provient du morceau *Amen, Brother* enregistré en 1969 par The Winstons. Ce passage syncopé et dépourvu d'instruments harmoniques a permis à de nombreuses personnes dans les années 90 et 2000 de sampler et looper ce rythme. Aujourd'hui il existe même un site qui lui est dédié : The Amen Break, et qui explique comment un pan entier de la musique électronique en découle.

Exemples de morceaux

Maintenant que vous avez vu comment ce langage fonctionne et comment on peut créer un sample, nous vous proposons d'écouter, en guise de conclusion, quelques morceaux que l'on a partiellement ou entièrement produits. Le premier, produit par eefano sur GitHub, a été retravaillé et adapté pour pouvoir utiliser notre piano. Il s'agit d'une reproduction très fidèle de la *Valse* de Dmitri Chostakovitch. On peut noter l'utilisation d'une fonction sinusoïdale pour faire fluctuer le tempo. Pour le second morceau, nous avons recréé de toute pièce une des hymnes de la house music : *Gypsy Woman* de Crystal Waters. Le but a été de se rapprocher le plus possible du son original en prêtant attention à la moindre percussion, ligne de basse et voicing des accords. Si vous voulez en entendre ou en apprendre plus, nous vous invitons à aller sur notre répertoire GitHub ou à directement regarder la documentation de Strudel qui est très fournie et utile.

Programme Python et tableaux, *fourier.py*

```python

import numpy as np
import matplotlib.pyplot as plt
from scipy.fftpack import fft
from scipy.io import wavfile
from matplotlib.backend_bases import MouseButton


def on_click(event):
    """
    programmmation événementielle : quand un événement se fait, une action est effectuée
    paramètre : événement considéré
    affiche la fréquence de l'endroit où la souris est cliquée
    """
    if event.button is MouseButton.RIGHT:
        print(f'fréquence : {event.xdata}')



def main():
    """
    applique les transformées de Fourier
    """
    fs, data = wavfile.read('la3.wav')
    piste = data.T[0]
    b=[(ele/2**8.)*2-1 for ele in piste] # this is 8-bit track, b is now normalized on [-1,1]
    #on divise par 2**8 pour convertir les données en bit
    transformee = fft(b)
    taille = len(transformee)/2
    """
    gère le visuel de la transformée avec les fonctions de matplotlib
    """
    plt.figure(figsize=(13,13))
    axes = plt.gca()
    axes.set(xlim = (0,1500), xticks = np.arange(0,1500,50))   #grosse graduation
    axes.set_xticks(np.arange(0,1500,10), minor=True)    #petite graduation
    plt.plot(transformee[:(int(taille-1))],'r') #trace la courbe
    plt.grid()
    plt.title('Faire un clic droit pour obtenir la fréquence.')
    """
    s'occupe de récupérer la fréquence des maximums
    """
    binding_id = plt.connect('button_press_event', on_click)

    plt.show()
    #affichage


if __name__ == '__main__':
    main()
```

CODE Fourier	EXPLICATIONS
import matplotlib.pyplot as plt	Pour l’aspect graphique
from scipy.fftpack import fft	Récupérer la transformée de Fourier
from scipy.io import wavfile	Pour ouvrir le fichier WAV
import numpy as np	Pour le max
plt.figure(figsize=(13,13))	Dimension de la figure
fs, data = wavfile.read('fichier.wav')	Lire le fichier .wav
piste = data.T[0]	Récupérer la 1ère piste audio si stéréo
b=[(ele/2**8.)*2-1 for ele in piste]	Passage des données en bits
transformee = fft(b)	Transformée de Fourier
d = len(transformee)/2	La moitié suffit car b est réel, donc le spectre est symétrique
plt.plot(abs(transformee[:(int(d-1))]),'r')	int nécessaire sinon la variable est un float et les float ne sont pas pris comme indice dans les listes
axes = plt.gca()	
axes.set(xlim = (0,1500), xticks = np.arange(0,1500,50))	Graduation tous les 50 entre 0 et 1500
axes.set_xticks(np.arange(0,1500,10), minor=True)	Petites graduations tous les 10, petites car précision minor
plt.grid()	Affiche la grille
def on_click(event):	Ce qui se passe quand la souris est cliquée sur la fenêtre mathplotlib
if event.button is MouseButton.RIGHT:	Quand on clique à droite : (à droite car à gauche pour zoomer dans la fenêtre)
print(f'coordonnées : fréquence : {event.xdata}')	Affiche la coordonnée x de la souris, soit la fréquence correspondante
binding_id = plt.connect('button_press_event', on_click)	Fonction plt.connect(id_event, fonction(event : Event)-> any) : appelle la fonction on_click si l'évènement est qu'un bouton est pressé
plt.show()	Afficher le spectre

CODE Spectre	EXPLICATION
nombre=int(input("Combien d'harmonique voulez-vous ? "))	Demande à l'utilisateur combien d'harmonique il souhaite et change la réponse de str à int
nom_fichier = r'C:\Users\Lucie OLIVE\Documents\scolarité\prépa\2A\projet\spectre_la3.txt'	Donne le chemin absolu vers le fichier à utiliser, le r avant la chaîne de caractères permet d'avoir une chaîne brute
with open(nom_fichier, 'r') as fichier:	Ouverture du fichier en mode écriture
tableau = {}	Création d'un dictionnaire pour récupérer les valeurs : clés = fréquences, valeurs = niveau sonore
for ligne in fichier:	Passage par toutes les lignes du fichier
contenu = ligne.split()	Séparation de chaque ligne en fonction des espaces
if len(contenu) != 2:	Toutes les lignes contiennent 2 éléments sauf la 1ère qui donne la « légende », que nous souhaitons éviter
pass	On saute la ligne
else :	Quand c'est pas le cas et qu'on a bien la fréquence et le niveau sonore (toutes les autres lignes)
freq=float(contenu[0])	Récupération de la fréquence de la ligne et changement de son type pour réel
niv = float(contenu[1])	Récupération du niveau sonore de la ligne et changement de son type pour réel
tableau[freq]=niv	Ajout de la fréquence et du niveau sonore associé dans le dictionnaire
max = max(tableau.values())	Recherche de la valeur max, donc de la fondamentale de la note
for indice in tableau:	Parcours du tableau pour trouver la fréquence correspondante au max
if tableau[indice]==max:	Test des valeurs pour trouver la fréquence associée
print(f'Le niveau sonore maximal est de {max} et il est atteint pour {indice} Hz.')	On affiche la valeur max et la fréquence associée, soit la fondamentale
tableau[indice] = -12345678912345	Nous souhaitons avoir les harmoniques suivantes, donc les max successifs, nous « retirons » donc le max pour avoir accès au suivant
for i in range(nombre):	Recherche des harmoniques successives demandées

max_tab = max(tableau.values())
for indice in tableau:
if tableau[indice]==max_tab:
print(f"L'harmonique numéro {i+1} est atteinte pour {indice} Hz.")
tableau[indice] = -12345678912345
	Répétition du même processus que pour la fondamentale, mais dans le print nous précisons i+1 car le décompte du i démarre à 0 et celui des harmoniques à 1, 0 étant la fondamentale

Relevé des décisions des réunions

Réunion 1

Objectif : Peut-on "copier" un instrument réel (piano/guitare) uniquement avec du code ?

1. Déjà réalisée

- **sons :** on a enregistré 3 octaves de piano.
- **analyse :** un script Python pour la TF avec affichage d'un graphe pour vérifier si on récupère bien les données souhaitées
- **création :** Strudel, un peu brouillon pour l'instant...

2. Pistes de travail

- **Comment ça sonne ? :** en additionnant plusieurs ondes simples (sinusoïdes) à différentes hauteurs, on peut imiter le grain du piano, mais c'est très long et compliqué d'après M.Pantel
    - *Piste :* tester le son avec seulement la fréquence de base, puis ajouter progressivement les harmoniques pour voir quand le son devient passable.
- **L'attaque :** on a évoqué durant la réunion la différence entre un clavier MIDI qui fait "on/off" et le vrai piano ou selon la vitesse et la force qu'on met sur les touches le son sonne différemment.
    - *Piste :* utiliser l'ADSR dans Strudel (c'est possible)
- humain et machine :
    - il semble (c'est même sûr) que les imperfections soient ce qui fait la beauté du morceau.
    - *Piste :* comparer nos sons de piano *Strudel* avec des tables de notes pro et/ou Aslan qui joue du piano.
- **L'enveloppe :**

3. Qui fait quoi ? (Rôles pressentis)

- **Lucie :** application de la TF, séquençage du signal.

- **Clément :** théorie de la TF, et définir un son.
- **Aslan :** prise de son(fini?) et théorie de la TF et définir un son, apporter sa culture du conservatoire.
- **Loup :** Strudel, ouverture sur d'autres méthodes de séquençage, histoire du séquençage et de l'algorave.
- **Victor :** Strudel, ouverture sur d'autres méthodes de séquençage, histoire du séquençage et de l'algorave.

N.B : Il peut être probant de comparer notre rapport à la musique entre jazzman, pianiste du conservatoire, néophyte... Et de nous questionner sur la place des algorithme et/ou de la volonté d'ultra précision qui tue la chaleur de l'instrument.

Suite : Prendre une seule note, passer le script Python dessus, et essayer de ressortir les 4 fréquences les plus fortes pour les injecter dans Strudel ?

Réunion 2

Objectif : Peut-on "copier" un instrument réel (piano/guitare) uniquement avec du code ?

1. Déjà réalisée

- **Sons :** 3 octaves ont été redécoupées et séquencées. Aslan a commencé sa partie sur la vulgarisation de la Transformée de Fourier.
- **Analyse :** deux scripts Python pour la TF faits par Lucie avec affichage d'un graphe pour vérifier si on récupère bien les données souhaitées et identifier la fondamentale à partir d'un fichier WAV. Un des deux programmes étant encore incompris et donnant des résultats inattendus, il a été envoyé à M.Pantel pour éclaircir ces points.
- **Rédaction :** Aslan et Lucie ont commencé la partie sur la vulgarisation de la Transformée de Fourier et l'explication du code. De son côté Clément a bien entamé sa partie sur le son et les

harmoniques constituant les signatures harmoniques des instruments. Loup a également continué sa rédaction sur le mouvement de l'algorave.

- **Création :** recherches sur les fonctions nécessaires à l'implémentation d'enveloppe ADSR. Import des notes de piano séquencées sur un serveur local permettant de les utiliser dans *Strudel web*. Problèmes rencontrés :

- Qualité du son altérée ce qui nous amène à privilégier l'utilisation de GitHub pour stocker ces fichiers WAV.

- Lors de la création de chanson, il était difficile de contourner le fonctionnement cyclique de *Strudel*. C'est pourquoi ont été envisagées des musiques telles que le *Boléro* de Ravel (Après des recherches, ce problème a été résolu sans aide particulière).

2. Questions à l'ordre du jour et réponses apportées

· **Y a-t-il un intérêt à enregistrer plus de sons ? (attaque différente, tenue différente…)**

Discussion sur les premiers synthés et le fait qu'il n'y avait pas beaucoup de notes et qu'ils modifiaient la fréquence après avoir enregistré 1 ou 2 octaves. Question sous-jacente : quelle serait la perte de données si on prenait moins de note et qu'on jouait sur les fréquences pour obtenir celles qui manquent ?

· **Quelle est la différence entre les fft (fast Fourier transformation) des différents modules Python (objectif d'améliorer la qualité des samples importés dans *Strudel*) ?**

Il existe des différences entre les fft des différents modules, qui ont été implantées avec des objectifs différents et donc des besoins différents. Il peut y avoir une différence de performance, de précision, le codage ne se fait pas forcément sur le même nombre de bits (32 ou 64). De plus, les différentes fonctions rappellent probablement la même fonction fft dans leur code ou s'appellent entre elles.

3. Pistes de travail et prochaines étapes :

· Les transformées de Fourier sont le début du deep learning, il y a des réseaux de neurones différents, on peut représenter chaque instrument par un réseau de neurones différent.

· Choix d'un morceau et implantation de celui-ci dans *Strudel*

· Début de la rédaction de parties définitives du rapport

· Réflexion sur un rendu visuel final (spectre, programme)

· Recherche sur la perception de la musique par le cerveau humain

Réunion 3

Nous avons posé des questions à Mr Pantel au sujet du rapport :

- Est-ce que l'on doit mettre les explications Strudel dans le rapport, dans le notebook ou les 2 ? Par rapport à la longueur du rapport, le notebook est hors cadre du rapport. Au final on a décidé de décrire brièvement cet aspect dans le rapport, et de surtout le développer dans le notebook.

- Est-ce que la partie sur la TES est suffisante ? Pour le numérique, c'est 80 % du social, hors IAG qui ont un aspect plus écologique à prendre en compte, tant que c'est un humain qui dirige l'IA, ce serait "ok", avant il y avait besoin de connaissances pour faire de la musique, ce n'est plus le cas maintenant, il faut regarder les coût par rapport aux bénéfices pour l'informatique et l'IA

- pour les images du code, les passer en blanc pour la lisibilité

- Comment faire pour la bibliographie et en quelle quantité ? Il faut citer les sites les plus intéressants, donner les sites "métas" (forum, etc.), dire explicitement qu'on ne va pas tout

citer, avoir du recul sur la difficulté de se documenter, le faire correctement sur une partie pour montrer qu'on sait le faire.

- mettre le lien vers le GitHub dans le rapport

Puis à propos du code Strudel :

- Comment ça se passerait si on lui donnait des fichiers mp3 au lieu de wav ? Ça devrait les convertir automatiquement.

- Problème sur mettre la voix sur la musique : dans *Strudel* ce sont des activités concurrentes (toutes en parallèle) si on ne les coordonne pas, elles vivent leur vie.

- Est-ce qu'on met les commentaires dans le code *Strudel* ? 2 choix : soit on ne le fait pas et on explique pourquoi (le code ne sert à rien sans et ne peut donc pas être réutilisé), si on veut qu'ils comprennent à la lecture, il en faut.

Il s'agit d'un projet exploratoire, tout ce qu'on fait ne mène pas quelque part et c'est normal.

Mr Pantel nous a également rappelé qu'il y a la soutenance orale à faire après le rapport.

- il va falloir faire un support pour la visualisation : Canva, faire des captures d'écran vidéo (usage du code) et son (partie de Clément) pour les explications, mettre la partition puis les notations jazz et enfin le code pour montrer les étapes.

- il va falloir s'entraîner à l'oral avant le passage pour le temps et le partage de la parole.

Réunion 4

Cette dernière réunion avec notre tuteur nous a permis de lui poser des questions sur la mise en page et la réalisation de notre rapport de projet.

Pour la bibliographie, il nous a conseillé de numéroter les sources dans la bibliographie et d'annoter le rapport, au lieu de noter le lien d'un site internet ou le titre d'un livre directement dans le rapport. Finalement, on a décidé de faire des liens cliquables afin de retrouver les sources dans la bibliographie.

On a également parlé du notebook et du fait que ce n'était pas équitable vis à vis des autres groupes de projet de faire un notebook qui complète le rapport, mais que c'était préférable de faire un notebook en annexe afin d'exécuter le code. On laissera un texte explicatif sur le notebook dans le rapport.

On a échangé sur l'affichage du code sur le rapport et des problèmes de HTML notre tuteur nous a dit que c'était normal avec HTML.

À propos des explications sur le code *Strudel*, il a dit que c'était suffisant et qu'il ne fallait pas que ça soit trop long pour ne pas dépasser la limite de page.